

I2C Protocol

Protocol Fundamentals & Physical Layer

1. **Open-Drain Logic:** Why does I2C use open-drain/open-collector outputs? How does this influence the way you write your RTL for the **SDA** and **SCL** pins?
2. **Pull-up Resistors:** What happens to the bus timing if the pull-up resistor value is too high? How does this affect the t_r (rise time) parameter?
3. **Start/Stop Conditions:** Define the Start and Stop conditions. Why are these the only times **SDA** is allowed to change while **SCL** is high?
4. **Multi-Master Arbitration:** Explain the "Wired-AND" arbitration logic. If two masters start transmitting simultaneously, how does one master "lose" the bus without corrupting the data of the winner?
5. **Clock Stretching:** How does a Slave device signal to the Master that it needs more time to process data? How do you implement the logic to detect this on the Master side?

RTL Design & Implementation

6. **Tri-state Buffer Modeling:** Write the Verilog code to model the bidirectional **SDA** pin.
7. **Synchronizers:** I2C is asynchronous to your SoC clock. How many flip-flops would you use to synchronize **SCL** and **SDA**? Why?
8. **Glitch Filtering:** The I2C spec requires a 50ns glitch filter. How would you implement a digital deglitch circuit in RTL?
9. **FSM Design:** Sketch a high-level Finite State Machine (FSM) for an I2C Master. What are the critical states?
10. **SDA Setup/Hold:** How do you ensure that **SDA** transitions only when **SCL** is low, while maintaining enough setup and hold time relative to the **SCL** rising edge?
11. **Sampling Edge:** At which point of the **SCL** high period should your RTL sample the **SDA** line to ensure maximum noise immunity?
12. **Bit-Banging vs. Controller:** When would you recommend a dedicated RTL controller over bit-banging via GPIO?
13. **Clock Dividers:** If your system clock is 200MHz and you need Fast Mode (400kHz), how would you generate the **SCL** clock to ensure a 50% duty cycle (or the specific t_{high}/t_{low} ratios required)?
14. **Addressing Logic:** How do you implement 10-bit addressing in RTL compared to standard 7-bit addressing?
15. **Acknowledge (ACK/NACK):** Describe the RTL logic flow when a Master receives a NACK after sending a slave address.

Advanced Features & Constraints

16. **High-Speed Mode (3.4 Mbps):** What is the "Master Code" and how does the RTL behavior change once the bus enters High-Speed mode?

17. **Repeated Start:** Why is a Repeated Start used instead of a Stop followed by a Start? How does your FSM handle this?
 18. **Bus Clear:** If a Slave is stuck pulling **SDA** low, how can the Master RTL attempt to recover the bus?
 19. **General Call:** How does your RTL identify a General Call address, and what is the typical response logic?
 20. **Device ID:** How is the 24-bit Device ID read sequence implemented in the controller?
-

Verification & Debugging

21. **CDC Issues:** What are the risks of sampling **SDA** and **SCL** directly without synchronization? Could it lead to a false Start/Stop detection?
 22. **Functional Coverage:** List three critical covergroups you would define for an I2C Master RTL.
 23. **Assertions (SVA):** Write a SystemVerilog Assertion to check that **SDA** does not change while **SCL** is high (except for Start/Stop).
 24. **Protocol Violation:** How should your RTL behave if it detects a Start condition in the middle of a data byte?
 25. **Data Integrity:** I2C doesn't have a built-in CRC. If you were adding a non-standard CRC layer in RTL, where would it sit in the FSM?
-

Timing & Synthesis

26. **False Paths:** Would you set a **false_path** on the I2C I/O pins? Why or why not?
 27. **Input Delay:** How do you constrain **set_input_delay** for **SDA** relative to **SCL** in your SDC file?
 28. **Drive Strength:** How does the drive strength of your I/O pad affect the maximum frequency you can achieve on the bus?
 29. **Power Domains:** If the I2C block is in an "Always-On" domain but the CPU is not, what isolation logic is required?
 30. **Area Optimization:** If you are extremely gate-count limited, would you use a synchronous or asynchronous reset for the I2C logic? Justify your choice.
-

Certainly. Here are the technical answers for a Chip Design/RTL interview context.

Protocol Fundamentals & Physical Layer

1. **Open-Drain Logic:** I2C uses open-drain to allow multiple devices to drive the same line without electrical contention (short circuits). In RTL, you never drive a "1" onto the bus. You drive a "0" to pull the line low and "Z" (high impedance) to let the pull-up resistor pull it high.
2. **Pull-up Resistors:** If the resistor value is too high, the τ time constant increases, causing the signal to rise slowly. This rounds off the edges, potentially violating t_r (rise time) specs and causing the receiver to misread data at high frequencies.

3. **Start/Stop Conditions:** **Start:** SDA goes High-to-Low while SCL is High. **Stop:** SDA goes Low-to-High while SCL is High. This uniqueness prevents data bits (which change when SCL is low) from being misidentified as control signals.
4. **Multi-Master Arbitration:** Masters monitor SDA while transmitting. If a Master intends to send a "1" (High-Z) but senses a "0" on the bus (driven by another Master), it realizes it has lost arbitration, immediately stops its driver, and switches to Slave/IDLE mode.
5. **Clock Stretching:** A Slave pulls SCL low and holds it. The Master's RTL must include logic to "sense" the SCL line after it releases it. If SCL remains low, the Master FSM must wait before proceeding to the next clock pulse.

RTL Design & Implementation

6. **Tri-state Buffer Modeling:**

7. Verilog

```
assign sda = (sda_out_en) ? 1'b0 : 1'bz;
```

```
assign sda_in = sda;
```

Note: sda_out_en is high when the logic wants to drive a '0'.

8. **Synchronizers:** Typically 2 flip-flops are used to prevent metastability. Since I2C is very slow (kHz) compared to SoC clocks (MHz), 2-stage synchronization is sufficient.
9. **Glitch Filtering:** A simple shift register or counter-based filter. For example, a 3-bit shift register where the output only changes if all 3 bits are the same (111 or 000).
10. **FSM Design:** States: IDLE, START, SEND_ADDR, GET_ACK, DATA_XFER, RX_ACK, STOP, WAIT_STRETCH.
11. **SDA Setup/Hold:** Use a higher-frequency system clock to create sub-intervals of the SCL period. Change SDA at the mid-point of the SCL Low period to maximize setup and hold margins.
12. **Sampling Edge:** Sample SDA at the mid-point of the SCL High period, where the signal is most stable and furthest from transitions.
13. **Bit-Banging vs. Controller:** Bit-banging (Software) is fine for slow, infrequent tasks. A dedicated RTL controller is needed for high-speed transfers, multi-master support, or when the CPU cannot afford the overhead of toggling pins manually.
14. **Clock Dividers:** Use a counter running on the system clock. For 400kHz from 200MHz, the period is 500 cycles. Set SCL low for 250 cycles and High for 250 cycles (adjusting slightly for rise/fall time requirements).
15. **Addressing Logic:** 10-bit addressing uses two bytes. The first byte is a special reserved pattern 11110xx (where xx are the 2 MSBs of the 10-bit address). The second byte contains the remaining 8 bits.
16. **Acknowledge (ACK/NACK):** If NACK is received, the Master FSM must transition to the STOP state or a REPEATED START state to terminate or restart the transaction.

Advanced Features & Constraints

16. **High-Speed Mode:** The Master sends a "Master Code" (00001xxx) in Fast-mode. Slaves do not ACK this. Once detected, the bus switches to high-speed timing, often requiring active pull-ups or different current sources.
 17. **Repeated Start:** Used to change direction (Write to Read) or talk to a different slave without giving up bus ownership. This prevents another Master from grabbing the bus between transactions.
 18. **Bus Clear:** The Master can toggle SCL up to 9 times. This "clocks" the Slave FSM out of whatever state it is stuck in until it releases SDA. Then the Master sends a Start/Stop.
 19. **General Call:** Address 0000000. It's used to address every device on the bus simultaneously. Devices that don't need it simply ignore (NACK) it.
 20. **Device ID:** A 24-bit read-only sequence. Master sends a Start + Reserved ID address + Write, then a Restart + Reserved ID address + Read, then clocks out 3 bytes of ID data.
-

Verification & Debugging

21. **CDC Issues:** Without synchronization, a transition on SDA exactly at the SoC clock edge could cause a "runt" pulse or metastability in the internal FSM, leading to a false "Start" detection.
 22. **Functional Coverage:** 1. All Slave addresses hit. 2. Arbitration loss in every state. 3. Clock stretching handled at least once.
 23. **Assertions (SVA):**
Code snippet

```
property p_stable_sda;  
  @(posedge clk) (scl_sync == 1) |-> $stable(sda_sync);  
endproperty
```
 24. **Protocol Violation:** The RTL should reset its internal FSM to the IDLE state and wait for a new valid Start condition.
 25. **Data Integrity:** Usually implemented as a Packet Error Code (PEC) byte at the end of the data, calculated using a CRC-8 polynomial.
-

Timing & Synthesis

26. **False Paths:** No. These are asynchronous but timing-critical relative to each other. You use set_max_delay or specific CDC constraints, but never false_path.
27. **Input Delay:** You constrain SDA relative to the SCL edge to ensure the internal logic sees the data meeting the required setup time (\$t_{su;DAT}\$).
28. **Drive Strength:** Higher drive strength allows for faster discharge of the bus capacitance (faster fall times), which is necessary for higher frequencies (Fast-mode Plus/High-speed).
29. **Power Domains:** Level shifters (if voltages differ) and isolation cells are required to ensure that when the CPU is off, the I2C pins don't float or leak power into the AON domain.
30. **Area Optimization: Asynchronous reset** is usually smaller as it doesn't require extra logic gates at the "D" input of the flip-flop, but it requires a synchronizer to de-assert cleanly.

RTL Design & FSM Logic

1. How do you detect a START condition in RTL?

Sample **SDA** on two successive clocks. A START is detected when **SCL** is high, the previous **SDA** sample was '1', and the current **SDA** sample is '0'.

2. Why is a 'Synchronous' FSM often preferred over a 'Transition' FSM for I2C?

A synchronous FSM running on a high-frequency system clock allows for digital filtering of glitches and precise control over setup/hold times that are impossible if you use **SCL** as a clock.

3. How would you implement the "Slave Address Match" logic for 7-bit addresses?

Shift the incoming **SDA** bits into a 7-bit register. Once the 8th bit (R/W) is received, compare the register value against a hardcoded or programmable **SLAVE_ADDR** parameter.

4. What is the "Bit Counter" and why is it needed in the RTL?

A 3-bit counter that tracks the number of bits transferred (0 to 7). It triggers the FSM to move from the **DATA** state to the **ACK** state after every 8 bits.

5. How do you handle a "Lost Arbitration" mid-byte?

The Master must immediately release **SDA** (set to High-Z) and **SCL**. The FSM should transition to an **IDLE** or **SLAVE** mode and set an interrupt flag for the CPU to handle the retry.

Bus Timing & Physical Layer

6. Explain the "Wired-AND" behavior in terms of RTL simulation.

In simulation, if one driver outputs **1'b0** and another **1'bz** on a **tri1** net, the result is **1'b0**. This is how arbitration is modeled.

7. How do you calculate the **THIGH** and **TLOW** for a 100kHz I2C clock if your system clock is 50MHz?

The period is 10μs. At 50MHz (20ns clock), you need 500 system clocks. You would typically count 250 clocks for **SCL** Low and 250 for **SCL** High.

8. What is "Setup Time for Repeated Start" ($t_{SU;STA}$)?

It is the delay between **SCL** going high and **SDA** going low for the restart. RTL must ensure this delay meets the I2C spec (e.g., 0.6μs for Fast Mode).

9. How does "Bus Free Time" (t_{BUF}) affect your Master FSM?

After a STOP condition, the Master must wait for t_{BUF} before it can issue a new START to ensure Slaves have reset their internal logic.

10. What is the impact of excessive bus capacitance?

It increases the rise time. If the RTL samples the "High" level too early, it might read a '0' instead of a '1' because the voltage hasn't crossed the V_{IH} threshold yet.

Advanced Protocol Features

11. What is SMBus, and how does its RTL differ from standard I2C?

SMBus is based on I2C but adds a "Timeout" feature. If SCL is held low for more than 35ms, the RTL must reset the interface.

12. How do you implement "Packet Error Checking" (PEC)?

Implement a CRC-8 LFSR (Linear Feedback Shift Register) with the polynomial $x^8 + x^2 + x^1 + 1$. It updates with every byte transferred.

13. What is "PMBus"?

An extension of SMBus used for power management. RTL-wise, it follows the same physical layer but uses specific command sets and often requires a "Group Command Protocol."

14. Explain "Master-Receiver" vs "Master-Transmitter" modes.

In Transmitter mode, the Master drives SDA. In Receiver mode, the Master releases SDA (High-Z) and generates SCL pulses to let the Slave drive the data.

15. How does a Slave handle an address it doesn't recognize?

The Slave simply remains in the IDLE state and leaves SDA high during the 9th clock cycle (NACK).

Verification & Constraints

16. How do you verify "Clock Stretching" in a testbench?

The Slave BFM (Bus Functional Model) should pull SCL low for a random number of cycles after the Master releases it, checking if the Master waits correctly.

17. What is a "False Start" and how can RTL prevent it?

A false start is a glitch on SDA while SCL is high. Digital filtering (sampling SDA 3-4 times) ensures the transition is stable before the FSM reacts.

18. Write a constraint for the I2C output delay.

`set_output_delay -clock sys_clk -max 10 [get_ports sda_o]` (Values depend on the specific `$t_{VD;DAT}` requirement).

19. Why is `set_input_transition` important for I2C pins?

Because I2C has slow edges. The synthesis tool needs to know the transition speed to calculate the correct delay through the input buffers.

20. How do you test "Multi-Master" collision in UVM?

Drive two Master sequences simultaneously with different addresses and verify that the one with the lower address (more zeros) wins the bus.

Hardware Implementation Details

21. What are "IO Isolation Cells" in the context of I2C?

If the I2C block is powered down, isolation cells prevent the pins from driving unknown voltages (X) into the rest of the chip.

22. How do you implement a "Software Reset" for the I2C IP?

A bit in a control register that, when set, forces the FSM to `IDLE`, clears all internal counters, and resets the status flags without toggling the hardware reset line.

23. Why use a "Pulse Generator" for the `SCL` line?

To ensure that even if the system clock frequency changes (DVFS), the `SCL` high/low pulses can be adjusted via registers to maintain protocol timing.

24. What is the "Slave Data Hold Time" requirement?

Slaves must often provide a minimum hold time (around 300ns) to ensure the Master samples correctly. RTL adds this delay before changing the output state.

25. How do you handle "Clock Domain Crossing" (CDC) for the Interrupt line?

Since the I2C FSM and the CPU might be on different clocks, the "Transaction Complete" interrupt signal must be passed through a bit-synchronizer.

Error Handling & Debug

26. What happens if `SDA` is stuck High?

The Master will receive a NACK for every address sent. The RTL should set a "No Response" error flag.

27. What happens if `SCL` is stuck High?

No transactions can occur. The Master RTL will wait indefinitely for the first clock edge or trigger a "Bus Busy" timeout.

28. How do you debug I2C using an Internal Logic Analyzer (ILA)?

Trigger on the START condition and capture SCL, SDA, and the current state of the FSM to see where the protocol is failing.

29. Describe the "Arbitration Lost" status bit.

It's a sticky bit in the status register that stays '1' until the CPU reads it, indicating that the last attempted transfer failed due to another master.

30. Why is "Slave-Only" RTL smaller than "Master/Slave" RTL?

A Slave-only design doesn't need clock generation logic, arbitration logic, or the complexity of initiating START/STOP conditions.